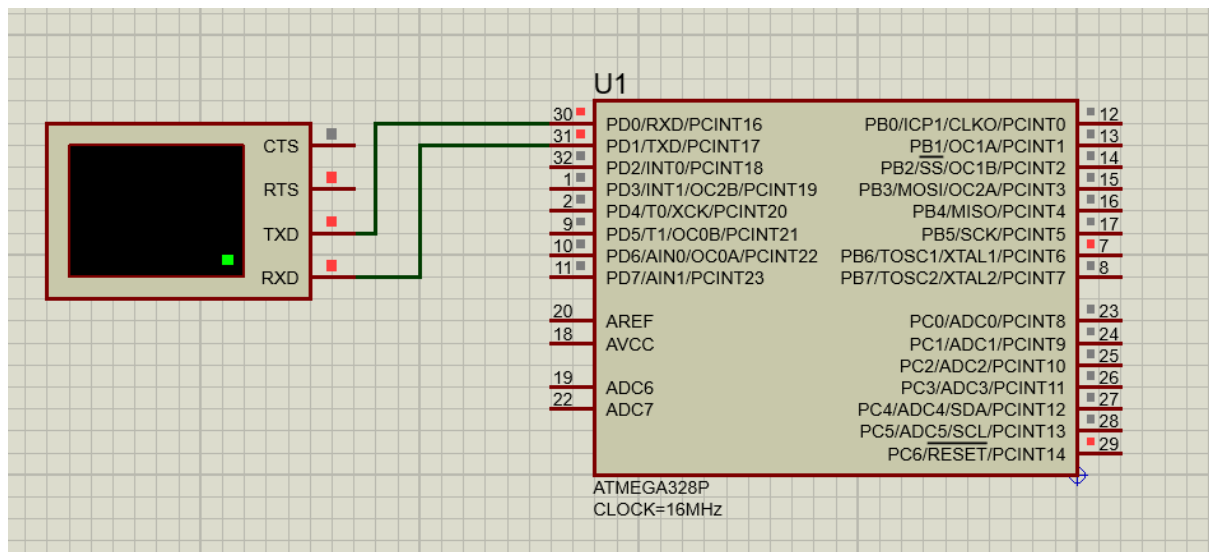


Ejercicio 2

Indicaciones Generales: La siguiente práctica puede realizarse sobre Proteus 8, simulando el microcontrolador ATmega328P. Configuración del microcontrolador en Proteus: componente ATMEGA328P, con Clock Frequency = 16 MHz y CKDIV8 sin programar. Terminal serie: Virtual Terminal de Proteus directamente a los pines PD0 (RXD) y PD1 (TXD) del microcontrolador, a 9600 baudios.

Desarrolle una aplicación de planificación que contenga 3 tareas. Las mismas deben competir por imprimir en la terminal un mensaje con el nombre de cada tarea. Realice pruebas variando las prioridades de cada una de las tareas, generando así diferentes escenarios en el mensaje de salida de la terminal.

para este ensayo planteamos un caso base usando freeRTOS, un ATMEGA328p a 16 MHz y una virtual terminal, con comunicación UART usando baudrate=9600

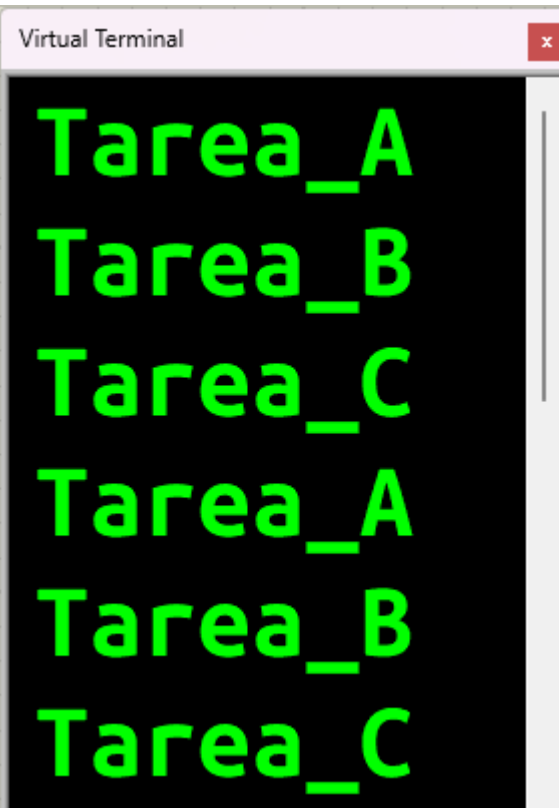


primer escenario

todas las tareas con prioridad= 1

primero se muestra la configuración y luego la salida por terminal virtual

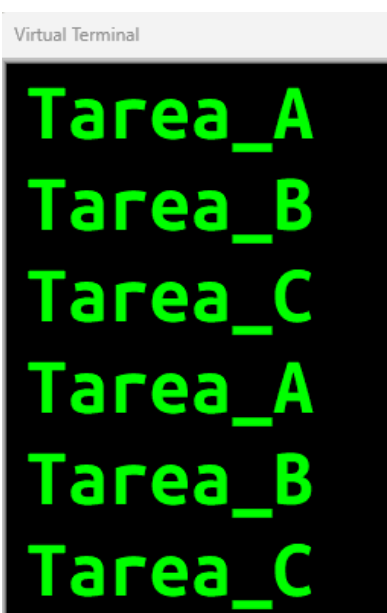
```
#define PRIO_A 1
#define PRIO_B 1
#define PRIO_C 1
```



The screenshot shows a 'Virtual Terminal' window with a black background and green text. The text displays the execution order of tasks A, B, and C. Since all tasks have a priority of 1, they are executed in a sequential order: Tarea_A, Tarea_B, Tarea_C, followed by a second iteration of Tarea_A, Tarea_B, and Tarea_C.

segundo escenario

```
#define PRIO_A 3
#define PRIO_B 2
#define PRIO_C 1
```



The screenshot shows a 'Virtual Terminal' window with a black background and green text. The text displays the execution order of tasks A, B, and C. Since tasks have different priorities (A=3, B=2, C=1), they are executed in descending order of priority: Tarea_A, Tarea_B, Tarea_C, followed by a second iteration of Tarea_A, Tarea_B, and Tarea_C.

tercer escenario

```
#define PRIO_A 1
#define PRIO_B 2
#define PRIO_C 3
```

```
Tarea_C
Tarea_B
Tarea_A
Tarea_C
Tarea_B
Tarea_A
```

cuarto escenario

```
#define PRIO_A 2
#define PRIO_B 3
#define PRIO_C 1
```

```
Tarea_B
Tarea_A
Tarea_C
Tarea_B
Tarea_A
Tarea_C
```

quinto escenario (no bloqueante)



```
#define PRIO_A 3
#define PRIO_B 2
#define PRIO_C 1

/* 0: la tarea imprime
 * 1: la tarea imprime
#define USE_DELAY 0
```

```
Tarea_A
Tarea_A
Tarea_A
Tarea_A
Tarea_A
Tarea_A
Tarea_A
Tarea_A
```

Conclusiones: al cambiar las prioridades, las tareas cambian el orden de ejecución. Además, en caso de no poner un delay luego de ejecutar la tarea (caso 5), la tarea A (primera y prioritaria) genera starvation en las otras tareas ya que nunca libera el recurso. En caso de tener la misma prioridad (caso 1) las tareas se ejecutan en el orden que fueron programadas.